

# Capture the Flag 5: Make the System Interactive

Goal: Transform your Student Directory into an interactive application where administrators can favorite students, edit information, and remove records using Flutter interactions and state management.

## Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

## Flag 0 — Create Your Feature Branch

Objective: Create a new feature branch before adding interactions.

Create:

`feature/student-interactions`

Screenshot:

```

User@DESKTOP-GL91V6M MINGW64 ~/Documents/Moreno_A3
$ git clone -b feature/student-list --single-branch https://github.com/AnthonySatoru/IT314AB_Moreno-A.git
Cloning into 'IT314AB_Moreno-A'...
remote: Enumerating objects: 239, done.
remote: Counting objects: 100% (239/239), done.
remote: Compressing objects: 100% (172/172), done.
remote: Total 239 (delta 37), reused 233 (delta 31), pack-reused 0 (from 0)
Receiving objects: 100% (239/239), 4.36 MiB | 32.00 KiB/s, done.
Resolving deltas: 100% (37/37), done.

User@DESKTOP-GL91V6M MINGW64 ~/Documents/Moreno_A3
$ cd IT314AB_Moreno-A

User@DESKTOP-GL91V6M MINGW64 ~/Documents/Moreno_A3/IT314AB_Moreno-A (feature/student-list)
$ git checkout -b feature/student-interactions
Switched to a new branch 'feature/student-interactions'

User@DESKTOP-GL91V6M MINGW64 ~/Documents/Moreno_A3/IT314AB_Moreno-A (feature/student-interactions)
$ git branch
* feature/student-interactions
  feature/student-list

User@DESKTOP-GL91V6M MINGW64 ~/Documents/Moreno_A3/IT314AB_Moreno-A (feature/student-interactions)
$

```

## 🚩 Flag 1 — Wake Up the Buttons

Add at least two buttons inside each Student Card.

Example actions:

- Favorite
- Edit

Objective: Make your Student Card respond to button presses using **onPressed**.

Right now your buttons are only decorations.

Research how Flutter's **onPressed** works.

The buttons do not need to perform their final behavior yet. They simply need to respond when pressed.

## Test

When a button is pressed, your app should visibly react in some way (such as printing a message or showing a temporary notification).

Screenshot:

```

Row(mainAxisAlignment:
  MainAxisAlignment.center,
  children: [
    InputChip(
      avatar: const Icon(
        Icons.favorite,
        size: 18,
      ),
      label: const Text('Favorite'),

      onPressed: (){
        print("Favorite pressed for ${profile.name}");
      }
    ),
    const SizedBox(height: 10),
    InputChip(

```



```

I/flutter (15405): Favorite pressed for Airi Sakura
I/flutter (15405): Favorite pressed for Airi Sakura
I/flutter (15405): Favorite pressed for Airi Sakura
I/flutter (15405): Favorite pressed for Airi Sakura
I/flutter (15405): Edit pressed for Airi Sakura

```

## 🚩 Flag 2 — Touch Anywhere

Objective: Make the entire Student Card tappable using `onTap`.

Buttons aren't the only interactive elements.

Research Flutter's `onTap`.

Wrap your Student Card so tapping anywhere on the card triggers an interaction.

## Test

The user should be able to:

- Tap the Favorite button

- Tap the Edit button
- Tap anywhere else on the Student Card

Each interaction should be recognized separately. (One function for the 2 buttons, another for the Student Card)

Screenshot:

```
return GestureDetector(  
  onTap: (){  
    print("Student Card tapped: ${profile.name}");  
  },  
);
```

```
I/flutter ( 5299): Edit pressed for Airi Sakura  
I/flutter ( 5299): Favorite pressed for Kakeru Ryuen
```

- `onTap` in your code
- Student Card being tapped

## 🚩 Flag 3 — Discover `setState`

Objective: Update the screen immediately using `setState`.

So far your app reacts. But does the UI actually change?

Research Flutter's `setState`.

Use it so pressing a button immediately updates something visible on the screen.

Examples include:

- Changing text
- Changing an icon
- Updating a counter

The important part is understanding that `setState` tells Flutter to rebuild the UI.

## Test

The screen should visibly change immediately after pressing the button.

Screenshot:

```
onPressed: (){
  setState(){
    profile.isFavorite = !profile.isFavorite;
  });
  print("Favorite pressed for ${profile.name}");
}
```

- `setState` in your code
- Before and after interaction

## 🚩 Flag 4 — Favorite Students

Objective: Let administrators mark students as favorites.

Schools often need to highlight important records.

Add a favorite feature that changes a student's appearance when marked.

Possible visual changes include:

- Filled heart/star
- Different card color
- Favorite label

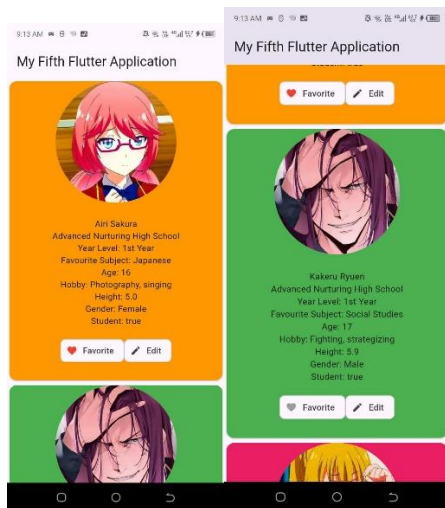
Each student's favorite status should work independently.

## Test

- Favorite one student.
- Other students should remain unchanged.
- Favorite another student.

Both should keep their own state.

Screenshot:



## 🚩 Flag 5 — Remove a Student

Objective: Delete a student from the directory.

Research how to remove an item from a Dart [List](#).

Add a Delete action that removes a student from your list.

The UI should immediately update after deletion.

## Test

Before:

- Student A
- Student B
- Student C

After deleting Student B:

- Student A
- Student C

No empty spaces should remain.

## Screenshot:



## 🚩 Flag 6 — Edit Trigger

Objective: Prepare your app for editing student information.

In the next CTF you'll build a full Edit Screen.

For now, create an Edit trigger.

Research simple ways Flutter can display temporary editing interfaces.

Examples include:

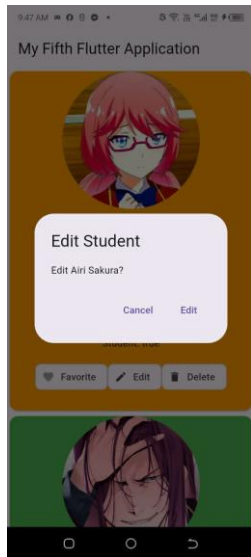
- `AlertDialog`
- `SnackBar`
- Bottom Sheet

The goal is simply to launch an edit action, not build the complete editor yet.

## Test

Pressing Edit should open your chosen interface.

Screenshot:



## 🚩 Flag 7 — Multiple Actions, Independent State

Objective: Make every Student Card manage interactions correctly.

This is your final interaction challenge.

Verify that every student behaves independently.

Example scenario:

- Favorite Student A.
- Delete Student C.
- Open Edit for Student B.

Your application should continue functioning correctly without affecting unrelated students.

### Test Checklist

- Favorite works independently.
- Delete updates the list immediately.
- Edit still opens correctly.
- Remaining students keep their own states.

Screenshot:



## 🚩 Flag 8 — Save Your Progress

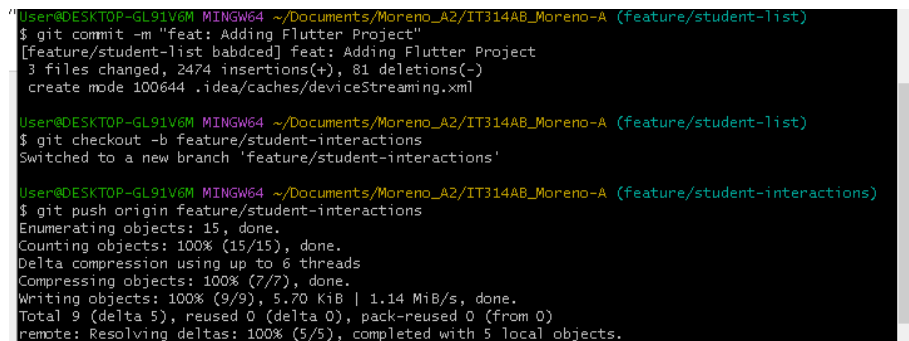
Objective: Commit and push your completed Interactive Student Directory.

Use a proper Git commit message.

Branch:

`feature/student-interactions`

Screenshot:



- Successful commit
- Current branch
- Terminal showing the commit